

AI Cognitive Operating System - Phase 1: Strategic Decision Support
ITAI 2376 Deep Learning in Artificial Intelligence Midterm Blueprint

Willie Breaux

April 6, 2026

1. Problem Statement

Many people struggle to make good decisions when they are dealing with too much information, multiple options, and unclear priorities. The problem is not always access to information, but turning that information into a clear decision. This problem affects students, professionals, creators, and anyone who has to research, compare options, and act under uncertainty.

My project will address this by building an AI Cognitive Operating System - Phase 1, focused on strategic decision support. The agent system will help a user take in a problem, organize the input, gather relevant information, reason through choices, and return a structured recommendation. This matters because this process could help users think more clearly and act more effectively.

2. Chosen Option

I am choosing **Option B: Multi-Agent System**. This problem fits a multi-agent design because strategic decision support naturally involves multiple stages, including intake, research, analysis, and final response. Using separate agents makes the system easier to organize and allows each agent to specialize in one part of the workflow.

3. Agent Architecture and Diagram

My system will use four agents: an intake Agent, a research agent, a reasoning agent, and a response agent. Each agent has a specific role and passes information to the next agent.

Agent 1: Intake Agent

- Input: User question or decision request
- Deep Learning Model: Transformer-based text classifier
- Agent Brain: LLM
- Tool: Input parser/file reader
- Output: Classified request type and cleaned task summary

Agent 2: Research Agent

- Input: Classified task summary from the intake Agent
- Deep Learning model: transformer embedding model for semantic retrieval
- Agent Brain: LLM
- Tool: Web search or document retrieval
- Output: gathered evidence and relevant findings

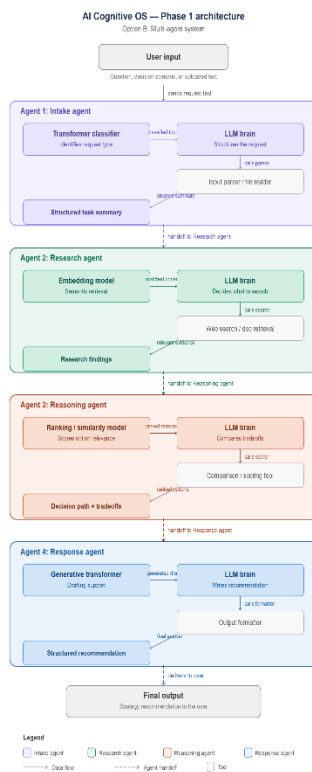
Agent 3: Reasoning Agent

- Input: Findings from Research Agent
- Deep Learning Model: Transformer-based ranking or similarity model
- Agent Brain: LLM
- Tool: Comparison/scoring logic
- Output: Ranked options, tradeoffs, and decision path

Agent 4: Response Agent

- Input: Ranked decision oath from Reasoning Agent
- Deep Learning Model: Generative transformer support
- Agent Brain: LLM
- Tool: Output formatter
- Output: Final recommendation for the user

Diagram Flow



4. Deep Learning Connection

My project connects to at least two course modules through its use of deep learning models. First, it uses transformer-based language models, which align with the project because the system must understand user requests, extract meaning from text, and generate coherent responses. A transformer is a strong choice because the project is language-heavy and depends on contextual understanding rather than simple keyword matching.

Second, the project uses embeddings and retrieval-based transformer methods to find relevant information and support reasoning. This fits the task because the system must compare ideas, retrieve useful context, and connect related information. Instead of reading everything manually each time, the retrieval component helps the system work more efficiently and focus on the most relevant content.

The LLM serves as the agent's brain, but the deep learning models do separate work, such as classification, retrieval, ranking, and generation. This separation follows the updated project instruction, which distinguishes the deep learning model from the LLM reasoning layer.

5. Agent Framework

I plan to use LangChain with LangGraph for the final build. I chose this framework because it supports tool calling, memory, multi-step orchestration, and structured agent flows. Since my project is a multi-agent system, LangGraph is a Strong fit because it allows workflows to be modeled as connected nodes and handoffs between agents.

I considered CrewAI because it is also designed for multi-agent collaboration. However, LangChain and LangGraph seem to be a better fit for my project because they offer greater flexibility in controlling reasoning, state flow, and tool integration. This will be one of my earliest serious uses of LangGraph, but I chose it because it aligns with the system design I want to build.

6. Tools and Data

Tools

1. **Web search tool** - needed so the Research Agent can gather outside information when the user's question requires current or broader context.
2. **Document retrieval/file reader** - needed to enable the system to read uploaded files, notes, or reference documents.
3. **Memory or context store** - needed to preserve user intent and maintain continuity across steps.
4. **Comparison/scoring tool** - needed so the Reasoning Agent can evaluate tradeoffs between choices.
5. **Output formatter** - needed to ensure the final recommendation is clearly structured for the user.

Data

The system will work with:

- user questions
- user-provided notes or documents
- public web information
- small test decision scenarios I create myself for development

I will likely create some of my own sample prompts and decision cases during testing so I can evaluate the system safely and clearly without privacy concerns. This makes the project easier to test while still demonstrating the full workflow.

7. Build Plan

Week 1 (April 6 - April 13): Finalize project scope, set up Langchain/LangGraph environment, and draft the first version of the multi-agent workflow.

Week 2 (April 14 -April 20): Built the intake Agent and Research Agent. Connect classification, retrieval, and at least one tool, such as a web search or file reading.

Week 3 (April 21 -April 27): Build the Reasoning Agent and Response Agent. Add ranking, comparison logic, and final answer formatting. test handoffs between agents.

Week 4 (April 28- May 3): Test the full pipeline end to end, improve reliability, record the demo, organize code files, and write the final reflection and documentation.

8. Expected Challenges

One challenge is making sure the system remains focused enough to complete it on time. The long-term vision for a Cognitive Operating System is broad, so I need to keep this Phase 1 version narrow and realistic.

A second challenge is separating the deep learning model's work from the LLM brain in a meaningful way. I need to make sure the non-LLM deep learning components are doing real work, such as classification, retrieval, or ranking, rather than making the whole project just one large LLM wrapper.

A third challenge is tool reliability. If the web search or retrieval step fails, downstream agents may receive less useful information. I will need fallback behavior and clear testing to ensure the system still delivers value even when a tool is limited.

A fourth challenge is the clarity of agent handoffs. Since this is a multi-agent system, I need to ensure each agent has a distinct role and that information flows smoothly from one stage to the next. If the handoffs are poorly designed, the system may become confusing or repetitive.

Resources

Choudhury, S. & Saravanan, R. (2025). "An integrative review on unveiling the causes and effects of decision fatigue to develop a multi-domain conceptual framework." *Frontiers in Cognition*.

<https://www.frontiersin.org/journals/cognition/articles/10.3389/fcogn.2025.1719312/full>

Devlin, J. et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *arXiv:1810.04805*. — the original BERT paper, cite this for your classifier.

LangChain (2024). "LangGraph: Multi-Agent Workflows." *LangChain Blog*.

<https://blog.langchain.com/langgraph-multi-agent-workflows/>

Reimers, N. & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *arXiv:1908.10084*. — the paper behind sentence-transformers and all-MiniLM-L6-v2, your embedding model.

Vaswani, A. et al. (2017). "Attention Is All You Need." *NeurIPS*. — the original transformer paper, foundational to everything in your project.

Yao, S. et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023*. — the reasoning acting loop your agents use.